

SECTION 5 — HASHMAP AND HASH-BASED COLLECTIONS

(Senior SDET Masterclass)

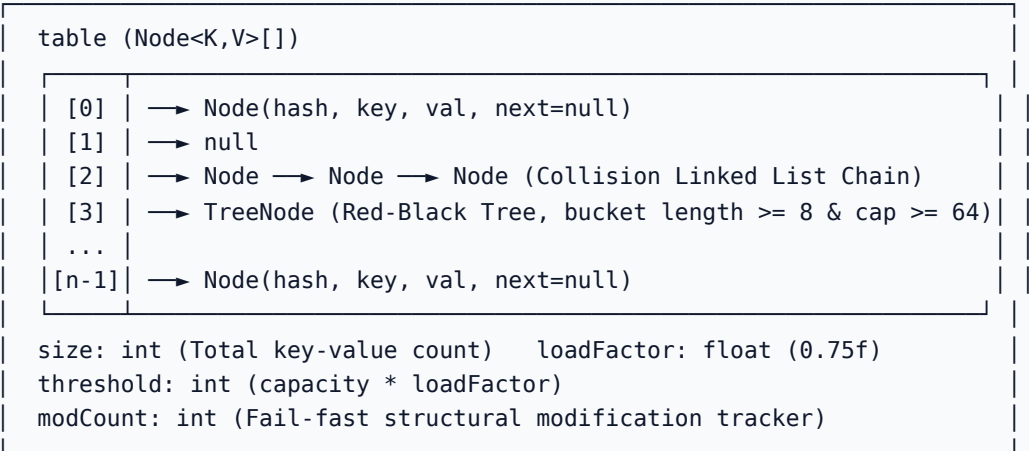
Topics Covered

- 5.1 Hashing Mental Model & Physical Memory Layout
- 5.2 `hashCode()` Contract & Distribution Entropy
- 5.3 Hash Spreading Function (`hash()` Bit-Shift Mechanics)
- 5.4 The Bucket Architecture (`Node<K, V>[] table` Array)
- 5.5 Bucket Index Calculation (`(n - 1) & hash` Bitwise Math)
- 5.6 Collision Resolution (Chaining vs. Open Addressing)
- 5.7 `equals()` Mechanics During Key Lookup
- 5.8 Updating an Existing Key (`putVal` Traversal)
- 5.9 Complete Lookup Flow (`getNode` Algorithm)
- 5.10 Removal Flow & Unlinking (`removeNode` Algorithm)
- 5.11 Resizing Mechanics (Power-of-Two Capacity Doubling)
- 5.12 Load Factor (The 0.75 Mathematical Equilibrium)
- 5.13 Treeification Mechanics (Red-Black Trees & Thresholds)
- 5.14 Time & Space Complexity (Average vs. Degenerate Worst-Case)
- 5.15 The Infamous Java 7 Infinite Loop vs. Java 8 Tail-Insertion Fix
- 5.16 High-Stakes Senior HashMap Interview Questions & Spoken Solutions
- 5.17 `HashSet` Internals (Adapter Pattern & Memory Footprint)

5.1 Hashing Mental Model & Physical Memory Layout

In the Java Virtual Machine, a `HashMap<K, V>` is implemented as an **Array of Hash Buckets**, where each bucket points to either `null`, a singly-linked list of `Node<K, V>` entries, or a balanced Red-Black Tree (`TreeNode<K, V>`).

HashMap Instance Memory Layout



The `Node<K, V>` Inner Class Structure

Every standard key-value mapping is stored in an instance of `HashMap.Node<K, V>` :

```
static class Node<K,V> implements Map.Entry<K,V> {
    final int hash;      // Cached 32-bit spread hash (avoids recomputing)
    final K key;         // Immutable key reference
    V value;             // Mutable value reference
    Node<K,V> next;     // Pointer to next collided node in the bucket
}
```

5.2 `hashCode()` Contract & Distribution Entropy

When an object is placed into a map, the JVM invokes `key.hashCode()` :

- It returns a signed 32-bit integer (`-2,147,483,648` to `+2,147,483,647`).
- **Ideal Hashing:** Maps uniform random inputs across the entire 4-billion integer spectrum.
- **Degenerate Hashing:** If `hashCode()` returns a constant (e.g., `return 1;`), every single entry collides into bucket index 1. The map degenerates from an $O(1)$ constant-time lookup into an $O(N)$ linear linked list search, devastating framework performance.

5.3 Hash Spreading Function (`hash()` Bit-Shift Mechanics)

In Java 8, `HashMap` does not use `key.hashCode()` directly. It runs it through an internal **hash spreading / perturbation function**:

```
// OpenJDK HashMap.java source code
static final int hash(Object key) {
    int h;
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >>> 16);
}
```

Why Bit-Shift Right by 16 and XOR (`^`)?

1. In a newly initialized `HashMap` , the table capacity is small ($n = 16$).
2. The bucket index is calculated using `(n - 1) & hash` (`$15 \& \text{hash} = 0b00001111 \& \text{hash}`).
3. Notice that **only the lowest 4 bits** determine which bucket the entry lands in! The top 28 bits are completely ignored by the bitwise AND mask.
4. If two keys produce hash codes that differ in their higher bits but share the same bottom 4 bits (e.g. `0x1004` and `0x2004`), they will collide 100% of the time.
5. **The Fix:** `h >>> 16` shifts the high 16 bits down into the lower half. The XOR (`^`) operation blends the entropy of the upper bits into the lower bits, preventing catastrophic clustering in low-capacity maps.

5.4 The Bucket Architecture & 5.5 Bucket Index Calculation

The Bitwise Index Calculation

Capacity (n) = 16	==>	Binary: 00000000 00000000 00000000 00010000
n - 1 = 15	==>	Binary: 00000000 00000000 00000000 00001111
Hash code	==>	Binary: 01011010 11001011 10010001 10100101
& (Bitwise AND)	==>	Binary: 00000000 00000000 00000000 00001111
Bucket Index	==>	Binary: 00000101 (Decimal: 5)

Why (n - 1) & hash Instead of hash % n ?

1. **CPU Efficiency:** The modulo operator (%) requires division, which takes approximately **20–40 CPU cycles**. Bitwise AND (&) executes in a **single CPU cycle (1 clock cycle)**.
2. **Negative Hash Code Immunity:** Modulo on negative numbers produces negative array indices (`-5 % 16 = -5`), requiring extra `Math.abs()` sanitization. Bitwise AND with (n - 1) (where n-1 is positive) mathematically guarantees the resulting index is strictly within `[0, n - 1]`.
3. **The Mathematical Requirement:** This bitwise equivalence **ONLY holds true if n is a power of two (2^k)**. This is why `HashMap` forces its capacity to be 16, 32, 64, 128 \dots at all times.

5.6 Collision Resolution & 5.7 equals() During Lookup

A collision occurs when two different keys produce hashes that resolve to the same bucket index:

```
Index 4: [Node: key="userA", hash=132] → [Node: key="userB", hash=84] → null
```

How getNode() Resolves Collisions

When you execute `map.get("userB")` :

1. Calculate `hash = hash("userB")`.
2. Locate bucket `index = (n - 1) & hash`.
3. Inspect the first node at `table[index]`.
4. Check if the key matches using the **Dual Equality Check**:

```
if (p.hash == hash && ((k = p.key) == key || (key != null && key.equals(k))))
```

- `p.hash == hash` : Quick primitive comparison. If hashes differ, keys CANNOT be equal.
 - `p.key == key` : Fast reference identity check.
 - `key.equals(k)` : Invoked only if reference identity fails.
5. If the first node does not match, traverse the linked list (or Red-Black tree) until a matching node is found or `null` is reached.

5.8 Updating an Existing Key vs. Inserting a New Key

```
// OpenJDK HashMap.putVal() logic flow:
1. If table is null or empty, resize() to allocate initial capacity 16.
2. If table[i] is null, create a new Node and place it directly into table[i].
3. If table[i] is NOT null (Collision):
  a. Check if first node matches the key. If yes, node found.
  b. Else if node is an instance of TreeNode, delegate to putTreeVal().
  c. Else (Linked List chain):
    - Traverse list counting nodes (binCount).
    - If matching key found, break loop.
    - If end of list reached (p.next == null):
      * Append new node to the TAIL (Java 8).
      * If binCount >= TREEIFY_THRESHOLD - 1 (8), trigger treeifyBin(table, hash).
4. If matching key was found:
  - Overwrite old value with new value.
  - Return oldValue.
5. Increment modCount and size.
6. If size > threshold, invoke resize().
```

5.11 Resizing & 5.12 Load Factor

The 0.75 Load Factor Balance

$\text{Threshold} = \text{Capacity} \times \text{Load Factor} \quad (16 \times 0.75 = 12)$

- Upon inserting the **13th element**, `HashMap` triggers a resize.
- Why 0.75?** It represents a mathematically proven trade-off between **time complexity** and **space overhead** based on the Poisson distribution:
 - If load factor is **1.0**: Saves memory, but bucket collisions increase exponentially, degrading lookup times.
 - If load factor is **0.5**: Extremely few collisions, but 50% of the allocated table array sits empty, doubling heap memory consumption.

The Java 8 Split-Pointer Resizing Optimization

In Java 7, resizing required re-hashing every single key.

In Java 8, because capacity doubles by a factor of 2 (e.g., from 16 to 32), an entry in bucket j can ONLY move to one of two locations:

- Stay at the **exact same index j** (`loHead / loTail`).
- Move to **index $j + \text{oldCapacity}$** (`hiHead / hiTail`).

```
// Java 8 tests the new bit without recomputing hash:
if ((e.hash & oldCap) == 0) {
    // Bit is 0: Stays at current index
    loTail.next = e;
} else {
    // Bit is 1: Moves to index + oldCap
    hiTail.next = e;
}
```

5.13 Treeification Mechanics (Red-Black Trees)

To protect applications against **Hash Collision Denial-of-Service (DoS) Attacks** (where attackers craft thousands of malicious strings producing identical hash codes to spike CPU to 100%), Java 8 introduced Treeification:

Bucket Length < 8

Singly-Linked List
O(N) linear search



Bucket Length ≥ 8 AND Table Capacity ≥ 64

Red-Black Tree
O(log N) search

The 3 Critical Threshold Constants

1. **TREEIFY_THRESHOLD = 8** : When a linked list in a bucket reaches 8 nodes, it becomes eligible to convert into a Red-Black Tree.
2. **MIN_TREEIFY_CAPACITY = 64** : If bucket length reaches 8, but the total table capacity is **less than 64**, the map **does NOT treeify**. Instead, it resizes (`resize()`), doubling the table to disperse the collided keys.
3. **UNTREEIFY_THRESHOLD = 6** : During resizing or removal, if the tree node count drops to 6, it converts back into a singly-linked list to reduce memory overhead (`TreeNode` consumes twice the memory of a standard `Node`).

5.14 Time and Space Complexity Master Reference

Operation	Best / Average Case	Degenerate Worst Case (Java 7)	Degenerate Worst Case (Java 8+)
get(key)	$O(1)$	$O(N)$ (pure linked list)	$O(\log N)$ (Red-Black tree)
put(k, v)	$O(1)$	$O(N)$	$O(\log N)$
remove(k)	$O(1)$	$O(N)$	$O(\log N)$
Space Overhead	$O(N)$	$O(N)$	$O(N)$ (\$32\text{ bytes/Node}\$; \$64\text{ bytes/TreeNode}\$)

5.15 The Infamous Java 7 Infinite Loop vs. Java 8 Tail-Insertion Fix

Why Java 7 Deadlocked During Concurrent `put()`

In Java 7, `HashMap.transfer()` utilized **Head-Insertion** during resizing:

1. Thread 1 and Thread 2 both trigger `resize()` concurrently.
2. Bucket contains nodes \$A \rightarrow B \rightarrow \text{null}\$.
3. Thread 1 gets paused right after reading \$e = A\$ and \$next = B\$.
4. Thread 2 completes resizing. Because of head-insertion, the pointers in the new table are **reversed**: \$B \rightarrow A \rightarrow \text{null}\$.

5. Thread 1 resumes execution with stale pointers. It attempts to insert `$A$` and then `$B$`, but `$B$`'s `next` already points back to `$A$`.
6. Result: **A Circular Reference (`$A$` to `B` to `A$`) is created.**
7. The very next `map.get()` on that bucket enters an infinite while-loop, pinning the CPU core at **100% utilization**.

How Java 8 Solved It

Java 8 abandoned head-insertion completely. Resizing now uses **Tail-Insertion** (`loTail` , `hiTail`), preserving the original pointer direction and making circular references impossible during resizing.

(Note: `HashMap` is still NOT thread-safe in Java 8; concurrent writes will still result in lost updates or corrupted tree states. Use `ConcurrentHashMap` for concurrency).

5.16 High-Stakes Senior HashMap Interview Questions & Spoken Solutions

Q1: "Can we insert a `null` key into a `HashMap` ? Where is it stored physically?"

*"Yes, `HashMap` allows exactly one `null` key and multiple `null` values.

Physically, when `key == null` , `HashMap.hash(null)` explicitly returns `0` . Because `0 & (capacity - 1) == 0` , a `null` key is **always guaranteed to be stored in bucket index `0`** of the internal `table[]` array.

In contrast, `Hashtable` , `TreeMap` , and `ConcurrentHashMap` do NOT permit `null` keys.

`ConcurrentHashMap` intentionally bans `null` keys and values to prevent ambiguity in concurrent environments where `map.get(key) == null` cannot reliably distinguish between 'key is absent' and 'key mapped to null' without non-atomic `containsKey()` checks."

Q2: "What is the optimal initial capacity if we expect to load 1,000 items into a `HashMap` without triggering resizing?"

"To store 1,000 items without triggering a resize, we must account for the default load factor of `0.75` :

$$\text{Required Capacity} = \left\lceil \frac{\text{Expected Elements}}{\text{Load Factor}} \right\rceil + 1 = \left\lceil \frac{1000}{0.75} \right\rceil + 1 = 1334 + 1 = 1335$$

Because `HashMap` always rounds up capacity to the nearest power of two, the constructor `new HashMap<>(1335)` will automatically adjust capacity to **2,048**.

If you set initial capacity to 1,000 directly (`new HashMap<>(1000)`), the table size would round to 1,024. Its threshold would be $1024 \times 0.75 = 768$. Inserting the 769th item would trigger an expensive resize in the middle of execution."

5.17 `HashSet` Internals

```

package com.deloitte.sdet.collections;

import java.util.HashMap;

/**
 * Conceptual representation of HashSet internals.
 */
public class ConceptualHashSet<E> {

    // HashSet is purely an Adapter wrapping an internal HashMap
    private transient HashMap<E, Object> map;

    // Dummy value paired with every element in the backing map
    private static final Object PRESENT = new Object();

    public ConceptualHashSet() {
        map = new HashMap<>();
    }

    public boolean add(E e) {
        // If put returns null, key did not exist -> returns true
        // If put returns PRESENT, key already existed -> returns false
        return map.put(e, PRESENT) == null;
    }

    public boolean contains(Object o) {
        return map.containsKey(o);
    }

    public boolean remove(Object o) {
        return map.remove(o) == PRESENT;
    }
}

```

Memory Consequence: A `HashSet<E>` has the exact same memory overhead as a `HashMap<E, Object>`, allocating a full `Node` instance and retaining the dummy `PRESENT` reference for every single entry.

5.5 High-Stakes Triage Scenario: High Collision Hash Flooding CPU Spike

Scenario Setup

A test data routing service experiences 100% CPU utilization when processing multi-tenant payloads due to hash collisions.

Senior Resolution Playbook

1. **Root Cause:** Poor `hashCode()` distribution maps thousands of entries to a single bucket index.
2. **Resolution:** Ensure high-entropy hash generation using `Objects.hash(field1, field2)` and leverage Java 8 Red-Black tree conversion ($O(\log N)$ tree nodes) by ensuring keys implement `Comparable`.

5.6 Anti-Patterns & Critical Pitfalls

- **Mutable Key Objects:** Mutating key state after Insertion causes silent `map.get(key) == null` lookups and memory leaks.
- **Null Key in Hashtable / ConcurrentHashMap:** Standard `HashMap` permits one `null` key, but `ConcurrentHashMap` throws `NullPointerException`.

5.8 Enterprise Relevance at 5,000+ Test Scale

At 5,000+ parallel test scale, high-throughput caching relies on `ConcurrentHashMap` segment locks to prevent thread contention.